# HW8 Step III — Database Comparison & Analysis

**Nithish Reddy Vemula** · CS 6650 Distributed Systems · March 22, 2026

## Part 0: Pre-Analysis Data Verification

Both test files contain exactly 150 operations each (50 create, 50 add, 50 get), confirmed programmatically. The combined dataset was generated by running `generate_combined.py` which merges `mysql_test_results.json` and `dynamodb_test_results.json` into `combined_results.json` with a metadata block verifying operation counts.

```
MySQL:    150 ops (50 create_cart, 50 add_items, 50 get_cart) — 150/150 success
DynamoDB: 150 ops (50 create_cart, 50 add_items, 50 get_cart) — 150/150 success
```

**Deliverable:** `loadTest/combined_results.json` — all numbers in this report are derived from this file.

## Part 1: Performance Comparison Table

### Overall Performance (from combined_results.json)

| Metric | MySQL | DynamoDB | Winner | Margin |
|---|---|---|---|---|
| Avg Response Time (ms) | 98.55 | 107.48 | MySQL | 8.93ms (9.1%) |
| P50 Response Time (ms) | 123.19 | 128.37 | MySQL | 5.18ms (4.2%) |
| P95 Response Time (ms) | 132.47 | 140.20 | MySQL | 7.73ms (5.8%) |
| P99 Response Time (ms) | 142.84 | 198.47 | MySQL | 55.63ms (38.9%) |
| Success Rate (%) | 100% | 100% | Tie | — |
| Total Operations | 150 | 150 | — | — |

*Data source: combined_results.json*

### Operation-Specific Breakdown

| Operation | MySQL Avg (ms) | DynamoDB Avg (ms) | Faster By |
|---|---|---|---|
| create_cart | 42.93 | 55.43 | MySQL by 12.50ms (29.1%) |
| add_items | 127.86 | 133.38 | MySQL by 5.52ms (4.3%) |
| get_cart | 124.87 | 133.65 | MySQL by 8.78ms (7.0%) |

**Analysis:** MySQL wins across every metric in the sequential 150-operation test. The gap is largest for `create_cart` (29%) because DynamoDB requires two operations (atomic counter UpdateItem + PutItem)

versus MySQL's single INSERT with AUTO_INCREMENT. The gap is smallest for `add_items` (4.3%) because both backends are dominated by the same bottleneck: two sequential network round-trips (MySQL: SELECT + INSERT/UPDATE inside a transaction; DynamoDB: GetItem + PutItem for read-modify-write).

The P99 divergence is the most significant finding — MySQL's P99 is 142.84ms (only 45% above its average) while DynamoDB's P99 is 198.47ms (85% above its average). DynamoDB has higher tail latency variance, likely due to adaptive capacity adjustments and partition management happening at the service level.

However, these sequential test numbers do not tell the full story. Under concurrent load (Locust tests), the picture changes at different scales.

## Locust Load Test Comparison (from Step I and Step II reports)

| Metric | MySQL (10u) | DynamoDB (10u) | MySQL (50u) | DynamoDB (50u) | MySQL (100u) | DynamoDB (100u) |
|---|---|---|---|---|---|---|
| RPS | 30.7 | 30.6 | 152.4 | 149.9 | 302.8 | 133.3 |
| Avg (ms) | 21.44 | 26.35 | 22.98 | 26.95 | 45.98 | 370.72 |
| Median (ms) | 20 | 26 | 22 | 27 | 22 | 330 |
| P95 (ms) | 25 | 31 | 26 | 32 | 31 | 840 |
| Max (ms) | 239 | 131 | 634 | 256 | 7,031 | 1,616 |
| Failures | 0% | 0% | 0% | 0% | 0% | 0% |

At 10 and 50 users, both databases perform nearly identically. The critical divergence is at 100 users: MySQL scales to 302.8 RPS while DynamoDB drops to 133.3 RPS (less than 50 users). DynamoDB's read-modify-write pattern for add_items creates serialization under contention. However, DynamoDB's worst-case latency (1.6s) is far better than MySQL's (7s), because MySQL's tail comes from connection pool exhaustion while DynamoDB's comes from distributed contention.

## Consistency Model Impact Assessment

**Observed behavior:** Across 50 consistency test iterations (20 read-after-write, 20 add-then-read, 10 rapid updates), DynamoDB showed 0% eventual consistency misses. Every read immediately returned the latest write. In practice, DynamoDB's within-region replication completes faster than the network round-trip from the application, making eventual consistency invisible for the shopping cart use case.

**ACID vs Eventual consistency comparison:**

| Aspect | MySQL (ACID) | DynamoDB (Eventual) |
|---|---|---|
| Read-after-write guarantee | Guaranteed | Not guaranteed (but observed 100% in practice) |
| Transaction isolation | Full (BEGIN/COMMIT/ROLLBACK) | None (single-item atomicity only) |
| Multi-item atomicity | Supported via transactions | Requires TransactWriteItems (expensive) |

| Aspect | MySQL (ACID) | DynamoDB (Eventual) |
|---|---|---|
| Lost update risk | None (row-level locks) | Possible with concurrent read-modify-write |
| Consistency cost | Included (no extra charge) | Strong reads cost 2× RCU |

**User experience implications:** For the shopping cart, eventual consistency had zero observable impact. A user adding items to their cart and refreshing the page will always see the updated cart because human interaction latency (100ms+) far exceeds DynamoDB's replication delay. The real risk is not stale reads but lost updates from concurrent writes — two users updating the same cart simultaneously could overwrite each other's changes under the current read-modify-write implementation.

---

# Part 2: Resource Efficiency Analysis

## Resource Utilization Comparison

**Connection management:**

MySQL requires explicit connection pool management. My Go server configured `MaxOpenConns=20`, `MaxIdleConns=10`, `ConnMaxLifetime=5min`. Under load, the CloudWatch metrics showed 11 active connections serving 302.8 RPS — efficient multiplexing, but misconfiguration (too few connections) would throttle throughput. DynamoDB has no connection concept — the AWS SDK manages HTTP connections internally via the default HTTP client. No pool sizing, no idle connection management, no connection lifecycle. This is significantly less operational complexity.

**Capacity planning:**

MySQL on `db.t3.micro` has hard limits: 150 max_connections, 1 vCPU, 1GB RAM, 20GB storage. Scaling requires provisioning a larger instance (vertical scaling) or adding read replicas (complex). DynamoDB on-demand has no capacity limits to manage — it auto-scales transparently. ConsumedWriteCapacityUnits peaked at ~9.17K during heavy load with zero throttling. No instance sizing, no storage provisioning, no replica management.

**Cost model:**

| Factor | MySQL (RDS db.t3.micro) | DynamoDB (On-Demand) |
|---|---|---|
| Base cost | ~$15/month (instance always running) | $0 when idle |
| Scaling cost | Instance upgrade ($$$) | Linear per-request ($1.25/million writes) |
| Provisioning time | 5-10 minutes | Instant |
| Teardown time | 3-5 minutes | Instant |
| Operational burden | Backups, patching, monitoring | Fully managed |

**How resources change with load:** MySQL resource consumption is step-function shaped — CPU and connections increase with load until the instance hits its ceiling, then you must vertically scale. DynamoDB consumption is linear — each request costs a fixed number of RCUs/WCUs, and the service scales

transparently. DynamoDB is more predictable for capacity planning because cost is directly proportional to traffic volume.

---

# Part 3: Real-World Scenario Recommendations

## Scenario A: Startup MVP (100 users/day, 1 developer, limited budget, quick launch)

**Recommendation: DynamoDB**

DynamoDB costs $0 when idle, compared to MySQL's ~$15/month minimum. At 100 users/day (~500 requests/day), the DynamoDB cost would be under $0.01/month. My testing showed identical performance at low load (10 users: DynamoDB avg 26ms vs MySQL avg 21ms — both imperceptible). Zero operational overhead means the solo developer doesn't need to manage backups, patching, connection pools, or schema migrations. DynamoDB tables create instantly via Terraform versus 5-10 minutes for RDS. The schemaless design allows rapid iteration without DDL changes.

## Scenario B: Growing Business (10K users/day, 5 developers, moderate budget, feature expansion)

**Recommendation: MySQL**

At 10K users/day with feature expansion, the application will need complex queries — customer purchase history, product analytics, inventory reports. MySQL's SQL layer (JOINs, GROUP BY, aggregations) supports these without application-level workarounds. My testing showed MySQL scales well to 50 concurrent users (152.4 RPS, 23ms avg) which comfortably handles 10K daily users. The connection pool model is well-understood by a 5-person team. Schema constraints (FKs, unique keys) prevent data integrity issues as multiple developers work on the codebase. Cost at this scale (~$15-50/month for RDS) is negligible relative to developer salaries.

## Scenario C: High-Traffic Events (50K normal, 1M spike, revenue-critical)

**Recommendation: DynamoDB for the spike-absorbing layer, MySQL for transactional core**

My testing revealed that DynamoDB handles load spikes without pre-provisioning — it absorbed 150 RPS at 50 users with zero throttling and no capacity planning. MySQL required RDS instance sizing and would need manual scaling for a 20× traffic spike. However, MySQL maintained 302.8 RPS at 100 users while DynamoDB degraded to 133.3 RPS due to read-modify-write contention. The hybrid approach: DynamoDB for shopping carts (simple access patterns, spike-tolerant) and MySQL for order processing (ACID transactions, complex queries needed for checkout, inventory deduction, payment processing).

## Scenario D: Global Platform (millions of users, multi-region, 24/7 availability)

**Recommendation: DynamoDB**

DynamoDB Global Tables provide built-in multi-region replication with single-digit millisecond latency worldwide. MySQL multi-region requires custom replication setup, conflict resolution, and significant operational expertise. My consistency test results (0% miss rate with eventual consistency) confirm that DynamoDB's replication model works for shopping carts. At millions of users, DynamoDB's on-demand auto-scaling eliminates the capacity planning nightmare that RDS would require across multiple regions. The cost-

per-request model (vs fixed instance cost per region) is also more efficient for globally distributed traffic with timezone-based peaks.

---

# Part 4: Evidence-Based Architecture Recommendations

## Shopping Cart Winner: MySQL

Despite DynamoDB's operational simplicity, I would choose **MySQL** for shopping carts based on my test results.

**Supporting evidence:**

- **Response time advantage:** MySQL was faster across all operations — 12.5ms faster on create (29%), 5.5ms faster on add (4.3%), 8.8ms faster on get (7.0%).
- **Throughput at scale:** MySQL achieved 302.8 RPS at 100 users versus DynamoDB's 133.3 RPS — a 2.3× advantage. This gap would widen at higher concurrency.
- **Atomic operations:** MySQL's `INSERT ... ON DUPLICATE KEY UPDATE` eliminates the lost-update risk present in DynamoDB's read-modify-write pattern. For a revenue-critical cart, data correctness is paramount.
- **Implementation complexity:** Despite having two tables and three indexes, the MySQL implementation was conceptually simpler — standard SQL operations versus DynamoDB's attribute value marshaling, condition expressions, and atomic counter pattern.

## When to Choose DynamoDB Instead

I would choose DynamoDB over MySQL when:

- **Traffic is highly variable** (event-driven spikes) — DynamoDB auto-scales without pre-provisioning, while MySQL requires instance right-sizing.
- **Operational team is small** — DynamoDB eliminates connection pool tuning, backup management, patching, and schema migrations.
- **Multi-region is required** — DynamoDB Global Tables are built-in versus MySQL's complex replication setup.
- **Cost at low scale matters** — DynamoDB is free when idle, MySQL costs ~$15/month minimum.
- **Access patterns are truly simple** — single-item reads/writes with no joins or aggregations.

## Polyglot Strategy for Complete E-Commerce

If building a full e-commerce system, I would use both:

| Component | Database | Rationale (from my findings) |
|---|---|---|
| Shopping carts | MySQL | Atomic upserts, 2.3× throughput advantage at scale, ACID for correctness |
| User sessions | DynamoDB | Simple key-value access, auto-expiry via TTL, no joins needed |
| Product catalog | MySQL | Complex queries (search, filter, sort by price/category), JOINs with categories |

| Component | Database | Rationale (from my findings) |
|---|---|---|
| Order history | MySQL | ACID transactions for checkout, JOINs with products/customers, reporting queries |
| Rate limiting | DynamoDB | High-write throughput, simple counters, auto-scaling for traffic spikes |
| Real-time notifications | DynamoDB | DynamoDB Streams for event-driven triggers, Lambda integration |

## Part 5: Learning Reflection

### What Surprised Me

**DynamoDB was slower than expected in the sequential test.** I assumed DynamoDB's managed infrastructure would be faster than a `db.t3.micro` MySQL instance, but MySQL won every metric. The overhead of two DynamoDB operations per create (counter + put) and two per add (get + put) versus MySQL's single-operation equivalents made a measurable difference.

**DynamoDB throughput degraded at high concurrency.** The throughput *decrease* from 149.9 RPS at 50 users to 133.3 RPS at 100 users was the most counterintuitive result. DynamoDB's "infinite scalability" applies to distributed access across many partition keys, not to contended access on individual items. The read-modify-write pattern on popular carts creates application-level serialization that DynamoDB cannot optimize away.

**Eventual consistency was never observed.** I expected to catch at least a few stale reads in 50 iterations, but DynamoDB's within-region replication was always faster than my test's read-after-write timing. This challenged my textbook understanding of "eventual consistency" — in practice, it's effectively immediate for single-region deployments.

**MySQL's tail latency was worse despite better average.** At 100 users, MySQL hit 7s max latency (connection pool exhaustion) while DynamoDB hit only 1.6s. MySQL's failure mode is a cliff — when the pool is full, all requests wait. DynamoDB's failure mode is gradual — contention slows individual operations but doesn't block unrelated requests.

### What Failed Initially

**RDS password with `@` character** — first `terraform apply` failed because AWS RDS prohibits `@` in passwords. Required a second deploy with a different password.

**AWS SDK version pinning** — I manually pinned AWS SDK v2 versions in `go.mod` that didn't exist in the registry, causing `go mod tidy` to fail with "unknown revision." The fix was stripping the versions and using `go get ...@latest` to let Go resolve them.

**204 No Content crash** — the performance test script assumed every HTTP response had a JSON body. The `POST /shopping-carts/{id}/items` endpoint returns 204 (no body), crashing the JSON parser. Required adding an empty-body check.

**Sequential test latency mismatch** — initial results showed 43-128ms for MySQL and 55-134ms for DynamoDB, but Locust showed 20-27ms for both. The gap was entirely due to Python urllib opening a new

TCP connection per request (no keep-alive). This taught me that test methodology can distort results as much as the actual database performance.

## Key Insights Gained

**When I would definitely choose MySQL:** Any application that needs complex queries (JOINs, aggregations, GROUP BY), multi-table transactions, or strong consistency guarantees. Also for workloads with known, steady traffic patterns where RDS instance sizing is predictable.

**When I would definitely choose DynamoDB:** Serverless or event-driven architectures, applications with extreme traffic variability, multi-region deployments, or simple key-value access patterns where operational simplicity outweighs query flexibility.

**What I would tell another student:** Run the Locust tests at multiple concurrency levels, not just the 150-operation sequential test. The sequential test showed a 9% difference between MySQL and DynamoDB — interesting but not dramatic. The Locust test at 100 users showed a 2.3× throughput difference and revealed fundamentally different scaling behaviors that the sequential test completely missed. The concurrent behavior is where SQL vs NoSQL really diverges.

**How hands-on implementation changed my understanding:** Before this assignment, I viewed SQL vs NoSQL as a schema flexibility question (structured vs schemaless). After implementation, I understand it's primarily an **access pattern and consistency model** question. DynamoDB's single-item atomicity is powerful for simple patterns but forces application-level complexity for anything multi-step. MySQL's query planner and transaction manager absorb that complexity at the database level. The "right" choice depends entirely on whether your access patterns align with DynamoDB's single-item model or need MySQL's relational capabilities.